ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



NHẬP MÔN TRÍ TUỆ NHÂN TẠO

BÁO CÁO BÀI TẬP LỚN 1

GIẢI BÀI TOÁN LOGIC PUZZLE PIPES

BẰNG GIẢI THUẬT TÌM KIẾM MÙ VÀ HEURISTIC

GVHD: TS. Vương Bá Thịnh
SV thực hiện: Hoa Toàn Hạc – 2210917
..... – .....

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

Mục lục

Lời mở đầu **2**

1 Giới thiệu và Mô hình hóa bài toán **3**
1.1 Mô tả trò chơi Logic Puzzle Pipes . . . 3
1.2 Biểu diễn Toán học bằng Hệ thống Bitmask 4-bit . . . 3
1.3 Mô hình hóa Bài toán Tìm kiếm (State Space Formulation) . . . 4
1.4 Đặc tả dữ liệu đầu vào và Thuật toán sinh bàn cờ có nghiệm . . . 5

2 Cấu trúc mã nguồn và Hiện thực giải thuật **6**
2.1 Kiến trúc tổng thể của hệ thống . . . 6
2.2 Giải thuật 1: Tìm kiếm mù – DFS Backtracking kết hợp Forward Pruning . . . 6
2.2.1 Nguyên lý hoạt động . . . 6
2.2.2 Mã giả thuật toán . . . 7
2.2.3 Đánh giá độ phức tạp lý thuyết của DFS . . . 7
2.3 Giải thuật 2: Tìm kiếm có thông tin – Thuật toán A^* Search . . . 8
2.3.1 Xây dựng hàm đánh giá $f(n)$ và hàm Heuristic $h(n)$ . . . 8
2.3.2 Chứng minh tính Chấp nhận được (Admissibility) và Ý nghĩa của Heuristic . . . 8
2.3.3 Mã giả thuật toán . . . 8
2.4 Phương pháp đo đạc Thời gian và Bộ nhớ tiêu thụ . . . 10

3 Hiện thực tính năng Demo trực quan Step-by-Step **11**
3.1 Các tính năng trên giao diện Web Visualizer . . . 11
3.2 Minh họa từng bước giải (Step-by-Step Trace) trên bàn cờ 3×3 . . . 11

4 Kết quả thực nghiệm và Đánh giá giải thuật **13**
4.1 Môi trường và Bộ dữ liệu kiểm thử . . . 13
4.2 Bảng số liệu thực nghiệm về Thời gian và Bộ nhớ . . . 13
4.3 Biểu đồ trực quan hóa kết quả thực nghiệm . . . 14
4.4 Phân tích và Giải thích kết quả thực nghiệm . . . 14
4.4.1 Giải thích về Số lượng nút đã duyệt (Nodes Explored) . . . 15
4.4.2 Giải thích về Thời gian thực thi (Execution Time) . . . 15
4.4.3 Giải thích về Sự tiêu tốn bộ nhớ (Peak Memory Consumption) . . . 15
4.4.4 Bảng tổng kết đối sánh hai giải thuật . . . 16

5 Kết luận và Hướng phát triển **17**
5.1 Kết luận . . . 17

Tài liệu tham khảo **18**

Lời mở đầu

Trong lĩnh vực Trí tuệ nhân tạo, các giải thuật tìm kiếm trong không gian trạng thái đóng vai trò nền tảng cho việc giải quyết các bài toán lập kế hoạch, tối ưu hóa tổ hợp và trò chơi logic. Thông qua việc biểu diễn một vấn đề thực tế thành các trạng thái, hành động chuyển tiếp và điều kiện đích, máy tính có thể tự động tìm kiếm và đưa ra lời giải hợp lệ mà không cần sự can thiệp trực tiếp của con người.

Trong khuôn khổ Bài tập lớn 1 môn Nhập môn Trí tuệ nhân tạo (HK1 năm học 2026–2027), nhóm lựa chọn nghiên cứu và giải quyết bài toán Puzzle Pipes trên hệ thống puzzle-pipes.com. Đây là bài toán thỏa mãn ràng buộc kết hợp với tính chất liên thông đồ thị toàn cục: mỗi thao tác xoay ống tại một ô không chỉ ảnh hưởng đến 4 ô láng giềng trực tiếp mà còn quyết định việc toàn bộ mạng lưới có bị phân mảnh hay tạo chu trình khép kín hay không.

Để giải quyết bài toán này theo đúng yêu cầu của đề bài, nhóm đã xây dựng một hệ thống hoàn chỉnh bằng ngôn ngữ Python 3, bao gồm các nội dung chính:

1. Mô hình hóa toán học bằng Bitmask 4-bit: Tối ưu hóa việc lưu trữ hướng của từng mảnh ống và biến phép xoay hình học 90° thành phép dịch bit tròn với độ phức tạp thời gian $O(1)$.
2. Hiện thực hai trường phái giải thuật tìm kiếm:
   - Tìm kiếm mù (Blind Search): Giải thuật tìm kiếm theo chiều sâu Depth-First Search (DFS) kết hợp quay lui (Backtracking) và kỹ thuật cắt tỉa nhánh sớm (Forward Pruning).
   - Tìm kiếm có thông tin (Heuristic Search): Giải thuật A^* Search sử dụng cấu trúc hàng đợi ưu tiên (Min-Heap) với hàm đánh giá $f(n) = g(n) + h(n)$ dựa trên số cổng xung đột cục bộ, thỏa mãn tính chất chấp nhận được (Admissible).
3. Trình sinh bàn cờ ngẫu nhiên luôn có nghiệm dựa trên thuật toán cây khung ngẫu nhiên (Randomized Spanning Tree), tạo bộ dữ liệu kiểm thử từ kích thước 3×3 đến 8×8.
4. Giao diện trực quan hóa tương tác (Web Visualizer Step-by-Step) xây dựng trên nền tảng Flask và HTML5/CSS3/SVG, cho phép theo dõi trực quan từng bước xoay ống, quay lui, điều chỉnh tốc độ phát và đối chiếu thông số hiệu năng.

Báo cáo dưới đây trình bày chi tiết quá trình tìm hiểu, mô hình hóa, thiết kế giải thuật, kết quả đo đạc thời gian thực thi – bộ nhớ tiêu thụ cùng những phân tích đánh giá của nhóm.

1 Giới thiệu và Mô hình hóa bài toán

1.1 Mô tả trò chơi Logic Puzzle Pipes

Puzzle Pipes (còn được gọi là Net hoặc FreeNet) là trò chơi giải đố logic trên trang web puzzle-pipes.com. Trò chơi diễn ra trên một bảng lưới hai chiều kích thước $R \times C$ (thường là lưới vuông $N \times N$). Tại mỗi ô (r, c) của lưới chứa một mảnh đường ống dẫn nước thuộc một trong năm hình dạng cơ bản:

- Ống cụt (End pipe): Chỉ có 1 cổng kết nối, đóng vai trò là điểm đầu hoặc điểm cuối của nhánh ống.
- Ống thẳng (Straight pipe): Có 2 cổng kết nối nằm đối diện nhau (180°).
- Ống góc vuông (Turn pipe): Có 2 cổng kết nối vuông góc với nhau (90°).
- Ống chữ T (Tee junction): Có 3 cổng kết nối tỏa ra 3 hướng.
- Ống chữ thập (Cross pipe): Có 4 cổng kết nối tỏa ra cả 4 hướng.

Ở trạng thái khởi đầu, tất cả các mảnh ống trên bàn cờ bị xoay ngẫu nhiên làm đứt gãy mạch nước. Người chơi hoặc tác tử AI chỉ được phép xoay tại chỗ từng mảnh ống theo các góc $90°, 180°, 270°$ mà không được hoán đổi vị trí giữa các ô. Trò chơi kết thúc thành công khi toàn bộ các mảnh ống trên bàn cờ thỏa mãn đồng thời 3 điều kiện:

1. Không hở biên: Không có bất kỳ cổng kết nối nào hướng ra ngoài ranh giới bức tường bao quanh bàn cờ.
2. Khớp nối hoàn toàn: Mọi cổng mở của một ô phải nối khớp trực tiếp với một cổng mở hướng ngược lại của ô láng giềng liền kề.
3. Liên thông toàn cục: Tất cả các ô có ống nước trên bàn cờ phải kết nối với nhau tạo thành duy nhất một mạng lưới liên thông (cây khung dẫn nước từ nguồn tới mọi điểm), không có bất kỳ cụm ống nào bị cô lập tách rời.

1.2 Biểu diễn Toán học bằng Hệ thống Bitmask 4-bit

Để tối ưu tốc độ kiểm tra ràng buộc trong quá trình tìm kiếm, nhóm biểu diễn mỗi mảnh ống bằng một số nguyên 4-bit (Bitmask) trong đoạn $[0, 15]$ tương ứng với 4 hướng địa lý: Bắc (NORTH), Đông (EAST), Nam (SOUTH) và Tây (WEST).

Quy ước trọng số bit như sau:

$$\text{Mask} = (b_3 b_2 b_1 b_0)_2 = b_3 \cdot 2^3 + b_2 \cdot 2^2 + b_1 \cdot 2^1 + b_0 \cdot 2^0 \tag{1}$$

Trong đó:

- Bit 0 (NORTH $= 1 = 0001_2$): Cổng nối lên phía trên $\Delta = (-1, 0)$, hướng đối diện là SOUTH (4).
- Bit 1 (EAST $= 2 = 0010_2$): Cổng nối sang bên phải $\Delta = (0, +1)$, hướng đối diện là WEST (8).
- Bit 2 (SOUTH $= 4 = 0100_2$): Cổng nối xuống phía dưới $\Delta = (+1, 0)$, hướng đối diện là NORTH (1).
- Bit 3 (WEST $= 8 = 1000_2$): Cổng nối sang bên trái $\Delta = (0, -1)$, hướng đối diện là EAST (2).



Mask = 1 (0001_2)	Mask = 10 (1010_2)	Mask = 3 (0011_2)	Mask = 6 (0110_2)	Mask = 11 (1011_2)	Mask = 15 (1111_2)
Ống cụt (N)	Ống thẳng (E-W)	Ống góc (N-E)	Ống góc (E-S)	Chữ T (N-E-W)	Chữ thập

Hình 1: Minh họa trực quan các loại mảnh ống và mã hóa Bitmask 4-bit tương ứng.

Khi xoay một mảnh ống 90° theo chiều kim đồng hồ (NORTH → EAST → SOUTH → WEST → NORTH), các bit ở vị trí $0, 1, 2$ dịch sang trái 1 bậc, còn bit thứ 3 (WEST) vòng về vị trí bit 0 (NORTH). Do đó, phép xoay 90° được tính trực tiếp bằng phép dịch bit tròn:

$$\text{RotateCW}(\text{mask}) = \big((\text{mask} \ll 1)\ \&\ 15\big)\ \mid\ (\text{mask} \gg 3) \tag{2}$$

Nhờ tính đối xứng hình học, số lượng hướng quay phân biệt của mỗi ô được rút gọn:

- Ống cụt, ống góc, ống chữ T: Có 4 trạng thái quay phân biệt.
- Ống thẳng ($\text{mask} \in \{5, 10\}$): Chỉ có 2 trạng thái phân biệt (dọc 5 và ngang 10), giúp giảm một nửa hệ số phân nhánh tại các ô này.
- Ống chữ thập ($\text{mask} = 15$) hoặc ô trống (0): Chỉ có 1 trạng thái duy nhất.

1.3 Mô hình hóa Bài toán Tìm kiếm (State Space Formulation)

Bài toán Puzzle Pipes trên lưới kích thước $R \times C$ được định nghĩa hình thức qua bộ 5 thành phần $\mathcal{P} = \langle \mathcal{S}, s_0, \mathcal{A}, \mathcal{T}, \mathcal{G} \rangle$:

1. Không gian trạng thái ($\mathcal{S}$): Một trạng thái $s \in \mathcal{S}$ được biểu diễn bởi cặp (k, M), trong đó:
   - $k \in \{0, 1, \ldots, R \times C\}$ là số lượng ô đã được cố định hướng theo thứ tự dòng quét từ trái sang phải, từ trên xuống dưới (chỉ số ô $i = r \times C + c$).
   - $M \in \{0, \ldots, 15\}^{R \times C}$ là ma trận cấu hình bitmask hiện tại của các mảnh ống trên bàn cờ.

   Nếu không áp dụng cắt tỉa, kích thước không gian trạng thái lý thuyết có thể lên tới $|\mathcal{S}| = \prod_{r,c} |\text{Rotations}(M_{r,c})| \leq 4^{R \times C}$ (với lưới 8×8, $4^{64} \approx 3.4 \times 10^{38}$ trạng thái).

2. Trạng thái ban đầu (s_0): Trạng thái $s_0 = (0, M_{\text{init}})$, trong đó chưa có ô nào được cố định hướng ($k = 0$) và M_{init} là ma trận bitmask đọc từ tệp dữ liệu đầu vào hoặc từ bộ sinh ngẫu nhiên.

3. Tập hành động hợp lệ ($\mathcal{A}(s)$) và Ràng buộc khớp cục bộ: Tại trạng thái $s = (k, M)$ tương ứng với ô hiện tại (r, c) (với $r = \lfloor k/C \rfloor, c = k \bmod C$), tập hành động $\mathcal{A}(s)$ là việc chọn một cấu hình xoay $m' \in \text{UniqueRotations}(M_{r,c})$ thỏa mãn điều kiện tương thích cục bộ:
   - Ràng buộc biên: Với mọi hướng $d \in \{\text{N, E, S, W}\}$, nếu $(m'\ \&\ d) \neq 0$ thì tọa độ láng giềng $(r + \Delta r_d, c + \Delta c_d)$ phải nằm trong phạm vi bàn cờ $0 \leq nr < R, 0 \leq nc < C$.
   - Ràng buộc khớp láng giềng đã gán: Với mọi ô láng giềng (nr, nc) đã được cố định hướng trước đó (ô phía trên $(r-1, c)$ và ô bên trái $(r, c-1)$), trạng thái cổng nối giữa hai ô phải khớp nhau:
   $$\big((m'\ \&\ d) \neq 0\big) \iff \big((M_{nr,nc}\ \&\ \text{OPPOSITE}[d]) \neq 0\big) \tag{3}$$

4. Hàm chuyển trạng thái ($\mathcal{T}(s, a)$): Khi thực hiện hành động gán hướng m' cho ô thứ k tại tọa độ (r, c), trạng thái chuyển từ $s = (k, M)$ sang trạng thái kế tiếp $s' = (k+1, M')$, trong đó $M'_{r,c} = m'$ và các ô khác giữ nguyên giá trị.

5. Kiểm tra trạng thái đích ($\mathcal{G}(s)$): Một trạng thái hoàn chỉnh $s = (R \times C, M)$ là trạng thái đích khi và chỉ khi:
   - Điều kiện 1 (Khớp cục bộ toàn bàn cờ): Mọi ô (r, c) không có cổng chỉ ra ngoài tường và khớp với cả 4 ô láng giềng xung quanh.
   - Điều kiện 2 (Liên thông toàn cục bằng BFS): Khởi chạy thuật toán duyệt theo chiều rộng (BFS) từ một ô có ống nước đầu tiên $(r_{\text{start}}, c_{\text{start}})$, đi theo các cổng kết nối mở. Số lượng ô thăm được $|\text{visited}|$ phải bằng đúng tổng số ô chứa ống nước trên bàn cờ. Điều kiện này loại bỏ các nghiệm giả bị tách thành nhiều vòng khép kín độc lập.

1.4 Đặc tả dữ liệu đầu vào và Thuật toán sinh bàn cờ có nghiệm

Các tệp dữ liệu đầu vào trong thư mục `inputs/` được lưu dưới định dạng văn bản thuần (`.txt`). Dòng đầu tiên chứa hai số nguyên dương R và C biểu thị số hàng và số cột; R dòng tiếp theo, mỗi dòng chứa C số nguyên trong đoạn $[1, 15]$ biểu diễn mã bitmask ban đầu của từng mảnh ống.

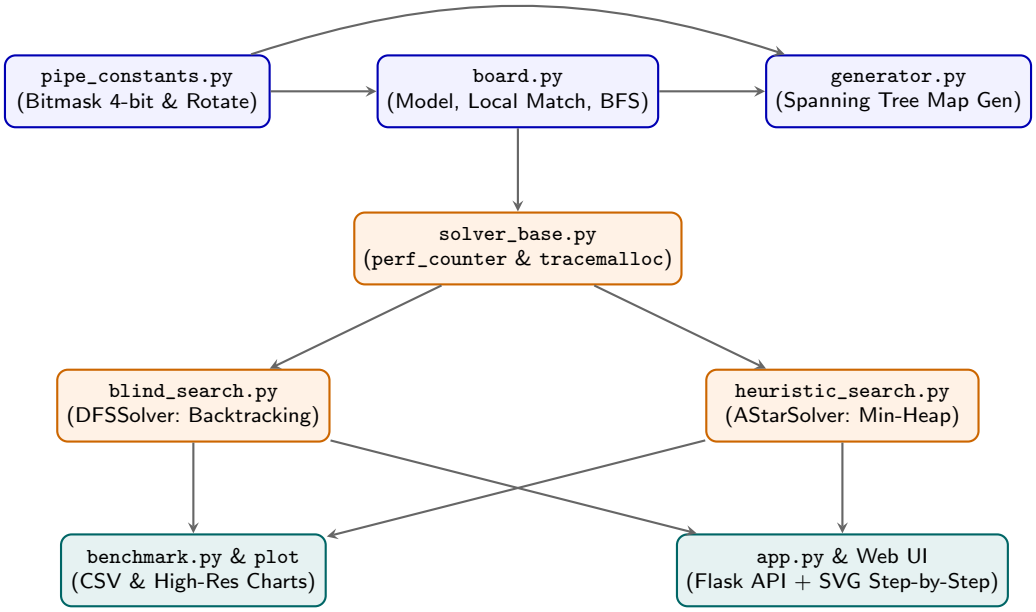
Để đảm bảo mọi bộ dữ liệu kiểm thử từ 3×3 đến 8×8 đều chắc chắn có lời giải hợp lệ, nhóm xây dựng thuật toán sinh bàn cờ dựa trên Cây khung ngẫu nhiên (Randomized Spanning Tree via DFS):

1. Khởi tạo lưới rỗng $R \times C$ với mọi ô có bitmask bằng 0.
2. Xuất phát từ ô $(0, 0)$, thực hiện duyệt DFS ngẫu nhiên (xáo trộn thứ tự 4 hướng ở mỗi bước). Mỗi khi đi từ ô (r, c) sang ô chưa thăm (nr, nc) theo hướng d, bật bit cổng d tại ô (r, c) và bật bit cổng đối diện OPPOSITE$[d]$ tại ô (nr, nc).
3. Khi toàn bộ $R \times C$ ô đã được thăm, ta thu được một cây khung liên thông hoàn chỉnh không chu trình làm nghiệm gốc.
4. Cuối cùng, xoay ngẫu nhiên mỗi ô từ 0 đến 3 lần để xáo trộn hướng, tạo ra bàn cờ đề bài.

2 Cấu trúc mã nguồn và Hiện thực giải thuật

2.1 Kiến trúc tổng thể của hệ thống

Hệ thống được thiết kế theo kiến trúc phân tầng, tách biệt giữa lõi giải thuật AI, bộ công cụ đo đạc thực nghiệm và giao diện trực quan hóa Web.



Hình 2: Sơ đồ kiến trúc các module trong dự án Puzzle Pipes AI.

Vai trò cụ thể của từng tệp mã nguồn trong dự án:

- `src/pipe_constants.py`: Định nghĩa hằng số hướng 4-bit, vector dịch chuyển $\Delta r, \Delta c$, phép xoay bit và hàm sinh các hướng quay không trùng lặp.
- `src/board.py`: Lớp `Board` quản lý lưới 2 chiều, đọc/ghi file, kiểm tra ràng buộc biên, kiểm tra khớp nối cục bộ, đếm xung đột và kiểm tra đích toàn cục bằng BFS.
- `src/solver_base.py`: Lớp cơ sở `SolverBase` chuẩn hóa giao diện giải thuật, bọc bộ đo thời gian thực thi (`time.perf_counter`) và bộ đo đỉnh bộ nhớ RAM (`tracemalloc`), đồng thời lưu vết từng bước xoay ống phục vụ trình chiếu Step-by-Step.
- `src/blind_search.py`: Hiện thực lớp `DFSSolver` kế thừa từ `SolverBase`.
- `src/heuristic_search.py`: Hiện thực lớp `AStarSolver` kế thừa từ `SolverBase`.
- `src/benchmark.py` và `src/plot_benchmark.py`: Tự động chạy thực nghiệm trên toàn bộ các tệp trong `inputs/`, xuất bảng tổng hợp ra file CSV và vẽ biểu đồ 300 DPI.
- `app.py`, `templates/index.html`, `static/js/app.js`: Máy chủ Flask và giao diện Web tương tác.

2.2 Giải thuật 1: Tìm kiếm mù – DFS Backtracking kết hợp Forward Pruning

2.2.1 Nguyên lý hoạt động

Trong bài toán Puzzle Pipes, một nghiệm hợp lệ luôn đòi hỏi mọi ô từ $(0, 0)$ đến $(R-1, C-1)$ đều phải được định hướng. Thay vì sử dụng tìm kiếm theo chiều rộng (BrFS) vốn gây bùng nổ bộ nhớ theo cấp số nhân ngay ở những hàng đầu tiên, nhóm lựa chọn giải thuật Tìm kiếm theo chiều sâu (DFS) kết hợp Quay lui (Backtracking) và kỹ thuật Cắt tỉa nhánh tức thời (Forward Pruning).

Giải thuật duyệt qua các ô theo thứ tự dòng quét: từ trái sang phải, hết hàng r mới chuyển xuống hàng $r+1$. Khi xét tại ô (r, c):

- Ô phía trên $(r-1, c)$ (nếu $r > 0$) và ô bên trái $(r, c-1)$ (nếu $c > 0$) đã được cố định hướng trong tập Assigned.
- Do đó, cổng Bắc của ô (r, c) bắt buộc phải khớp với cổng Nam của ô $(r-1, c)$, và cổng Tây của ô (r, c) bắt buộc phải khớp với cổng Đông của ô $(r, c-1)$.
- Đồng thời, nếu (r, c) nằm ở cạnh biên của bàn cờ $(r = 0, r = R-1, c = 0, c = C-1)$, không được có cổng nào hướng ra ngoài tường.

Nhờ kiểm tra tính tương thích cục bộ ngay trước khi gọi đệ quy, hầu hết các hướng xoay sai tại ô (r, c) đều bị cắt tỉa ngay lập tức (thường chỉ còn 0 hoặc 1 hướng hợp lệ được phép đi tiếp). Khi đi đến ô cuối cùng $(r = R)$ nhưng toàn mạng lưới chưa liên thông thành 1 thành phần duy nhất, giải thuật sẽ quay lui về các ô phân nhánh trước đó để thử cấu hình khác.

2.2.2 Mã giả thuật toán

Thuật toán 1 trình bày chi tiết từng bước của giải thuật DFS Backtracking kết hợp Forward Pruning:

Đầu vào: Bàn cờ B kích thước $R \times C$, tọa độ ô hiện tại (r, c), tập các ô đã gán hướng Assigned.
Đầu ra : True nếu tìm thấy cấu hình đích hợp lệ, ngược lại trả về False.

Hàm `Backtrack`$(B, r, c, Assigned)$**:**
  if $r \geq R$ **then**
    return IsGoalState(B) `// Kiểm tra khớp toàn cục và liên thông BFS`
  end
  if $c + 1 < C$ **then**
    $(next_r, next_c) \leftarrow (r, c+1)$
  else
    $(next_r, next_c) \leftarrow (r+1, 0)$
  end
  $m_{\text{orig}} \leftarrow B.\text{grid}[r][c]$
  Rotations $\leftarrow$ GetUniqueRotations(m_{orig})
  foreach $m_{cand} \in Rotations$ **do**
    nodes_explored $\leftarrow$ nodes_explored $+ 1$
    `// Forward Pruning: Kiểm tra khớp biên và khớp với ô (r-1, c), (r, c-1)`
    if *CheckLocalMatch*$(B, r, c, m_{cand}, Assigned)$ **then**
      $B.\text{grid}[r][c] \leftarrow m_{\text{cand}}$
      Assigned $\leftarrow$ Assigned $\cup \{(r, c)\}$
      RecordStep$(r, c, \text{“place”}, m_{\text{orig}}, m_{\text{cand}})$
      if `Backtrack`$(B, next_r, next_c, Assigned)$ **then**
        return True
      end
      `// Quay lui (Backtrack) khi nhánh hiện tại dẫn đến ngõ cụt`
      Assigned $\leftarrow$ Assigned $\setminus \{(r, c)\}$
      $B.\text{grid}[r][c] \leftarrow m_{\text{orig}}$
      RecordStep$(r, c, \text{“backtrack”}, m_{\text{cand}}, m_{\text{orig}})$
    end
  end
  return False

Algorithm 1: Giải thuật tìm kiếm mù DFS Backtracking kết hợp Forward Pruning.

2.2.3 Đánh giá độ phức tạp lý thuyết của DFS

- Độ phức tạp thời gian: Trong trường hợp xấu nhất lý thuyết là $O(b^N)$ với hệ số phân nhánh trung bình $b \leq 4$ và tổng số ô $N = R \times C$. Tuy nhiên, nhờ kỹ thuật Forward Pruning kiểm tra khớp nối với 2 ô đã gán là $(r-1, c)$ và $(r, c-1)$, hệ số phân nhánh hiệu dụng giảm mạnh xuống xấp xỉ $b_{\text{eff}} \approx 1.05 - 1.25$, giúp thời gian thực tế gần như tuyến tính theo số ô.
- Độ phức tạp không gian: Giải thuật thao tác trực tiếp trên ma trận bàn cờ và chỉ lưu ngăn xếp đệ quy có độ sâu tối đa $N = R \times C$. Do đó độ phức tạp bộ nhớ là $O(R \times C)$.

2.3 Giải thuật 2: Tìm kiếm có thông tin – Thuật toán A^* Search

2.3.1 Xây dựng hàm đánh giá $f(n)$ và hàm Heuristic $h(n)$

Thuật toán tìm kiếm A^* duy trì một tập mở (OpenSet) được cài đặt bằng cấu trúc dữ liệu Hàng đợi ưu tiên (Min-Heap). Tại mỗi bước lặp, A^* luôn trích xuất trạng thái n có giá trị hàm đánh giá $f(n)$ nhỏ nhất để mở rộng:

$$f(n) = g(n) + h(n) \tag{4}$$

Trong đó:

- Chi phí đường đi thực tế $g(n)$: Là số lượng ô đã được cố định hướng thành công tính từ ô $(0, 0)$ đến trạng thái hiện tại n:
$$g(n) = k = r \times C + c + 1 \tag{5}$$
- Hàm ước lượng Heuristic $h(n)$: Ước lượng chi phí còn lại để đi từ trạng thái n tới trạng thái đích hoàn chỉnh. Nhóm thiết kế hàm $h(n)$ dựa trên số lượng cổng kết nối đang bị lỗi (xung đột) trên toàn bàn cờ tại thời điểm hiện tại:
$$h(n) = \left\lceil \frac{\text{CountConflicts}(n)}{2} \right\rceil \tag{6}$$
Với CountConflicts(n) là tổng số cổng kết nối đang bị lỗi trên bàn cờ (bao gồm cổng đâm ra ngoài biên tường hoặc cổng mở nhưng ô láng giềng không có cổng đối ứng). Về mặt tính toán số nguyên, giá trị trần được tính chuẩn xác bằng công thức: $\lfloor(\text{CountConflicts}(n) + 1)/2\rfloor$.

2.3.2 Chứng minh tính Chấp nhận được (Admissibility) và Ý nghĩa của Heuristic

Một hàm Heuristic $h(n)$ được gọi là **chấp nhận được** (admissible) nếu với mọi trạng thái n, ta luôn có $h(n) \leq h^*(n)$, trong đó $h^*(n)$ là chi phí thực tế tối ưu để đi từ trạng thái n đến trạng thái đích. Các tính chất toán học của $h(n)$ được chứng minh như sau:

- **Không vượt ước ($h(n) \leq h^*(n)$):** Xét tại trạng thái n, mỗi thao tác gán và xoay hướng một ô (r, c) trong tương lai chỉ có thể giải quyết tối đa 2 cổng xung đột (cổng hướng sang ô bên trái $(r, c-1)$ và cổng hướng lên ô phía trên $(r-1, c)$ đã được gán trước đó). Do đó, để khử triệt để CountConflicts(n) cổng lỗi còn lại, giải thuật bắt buộc phải thực hiện thêm ít nhất $\lceil \text{CountConflicts}(n)/2 \rceil$ bước gán nữa. Vì chi phí đường đi được tính bằng số bước gán ô, ta luôn có:
$$h(n) = \left\lceil \frac{\text{CountConflicts}(n)}{2} \right\rceil \leq h^*(n) \tag{7}$$
Điều này khẳng định hàm Heuristic $h(n)$ không bao giờ đánh giá quá cao (never overestimates) chi phí thực tế, bảo đảm tính chấp nhận được của A^*.
- **Tại trạng thái đích n_{goal}:** Khi toàn bộ các ô đã được định hướng hoàn hảo, không còn bất kỳ cổng xung đột hay đâm vào tường nào (CountConflicts$(n_{\text{goal}}) = 0$), ta có:
$$h(n_{\text{goal}}) = \left\lceil \frac{0}{2} \right\rceil = 0 \tag{8}$$
- **Cơ chế ưu tiên nhánh có triển vọng:** Khi xét một ô (r, c) có nhiều hướng xoay hợp lệ cục bộ với $(r-1, c)$ và $(r, c-1)$, hướng xoay nào vô tình khớp sẵn với cả các ô chưa gán ở phía phải $(r, c+1)$ và phía dưới $(r+1, c)$ sẽ làm CountConflicts(n) giảm ngay lập tức. Nhờ phần tử trong Min-Heap được sắp xếp theo khóa $(f(n), h(n))$, A^* sẽ ưu tiên mở rộng ngay hướng xoay tạo ít xung đột tương lai nhất, từ đó giảm đáng kể số lượng nút phải duyệt so với tìm kiếm mù DFS.

2.3.3 Mã giả thuật toán

Thuật toán 2 mô tả chi tiết quy trình tìm kiếm của giải thuật A^* sử dụng hàng đợi ưu tiên Min-Heap:

Đầu vào: Bàn cờ khởi tạo B_{init} kích thước $R \times C$.
Đầu ra : Lịch sử bước giải History và trạng thái đích nếu thành công, ngược lại trả về False.

Hàm `AStarSearch`(B_{init})**:**
- $N \leftarrow R \times C, \quad M_0 \leftarrow B_{\text{init}}.\text{grid}, \quad h_0 \leftarrow \lceil \text{CountConflicts}(M_0)/2 \rceil$
- OpenSet $\leftarrow$ MinHeap(), Visited $\leftarrow \emptyset$
- Push(OpenSet, $\langle f = h_0, h = h_0, k = 0, M = M_0, \text{History} = \emptyset \rangle$)
- **while** *OpenSet* $\neq \emptyset$ **do**
  - $\langle f, h, k, M, \text{History} \rangle \leftarrow$ PopMin(OpenSet)
  - nodes_explored $\leftarrow$ nodes_explored $+ 1$
  - **if** $k == N$ **then**
    - **if** *IsGoalState*(M) **then**
      - **return** (True, M, History)
    - **end**
    - **continue**
  - **end**
  - $r \leftarrow \lfloor k/C \rfloor, \quad c \leftarrow k \bmod C$
  - Assigned $\leftarrow \{(\lfloor i/C \rfloor, i \bmod C) \mid 0 \leq i < k\}$
  - Rotations $\leftarrow$ GetUniqueRotations($M[r][c]$)
  - **foreach** $m_{cand} \in$ *Rotations* **do**
    - **if** *CheckLocalMatch*(M, r, c, m_{cand}, *Assigned*) **then**
      - $M' \leftarrow \text{CopyGrid}(M), \quad M'[r][c] \leftarrow m_{\text{cand}}$
      - **if** $(k+1, M') \notin$ *Visited* **then**
        - Visited $\leftarrow$ Visited $\cup \{(k+1, M')\}$
        - $g' \leftarrow k+1, \quad h' \leftarrow \lceil \text{CountConflicts}(M')/2 \rceil$
        - $f' \leftarrow g' + h'$
        - History$'$ $\leftarrow$ History $\cup \{\text{Step}(r, c, M[r][c], m_{\text{cand}})\}$
        - Push(OpenSet, $\langle f', h', k+1, M', \text{History}' \rangle$)
      - **end**
    - **end**
  - **end**
- **end**
- **return** False

Algorithm 2: Giải thuật tìm kiếm có thông tin A^* Search với hàng đợi ưu tiên Min-Heap.

2.4 Phương pháp đo đạc Thời gian và Bộ nhớ tiêu thụ

Để đảm bảo tính khách quan khi so sánh hai giải thuật, cả `DFSSolver` và `AStarSolver` đều được đo đạc thông qua cùng một cơ chế chuẩn hóa trong lớp cơ sở `SolverBase`:

- Đo thời gian thực thi (Execution Time – ms): Sử dụng bộ đếm độ phân giải cao `time.perf_counter()` của Python, loại bỏ sai số do lập lịch hệ điều hành và chuyển đổi sang đơn vị mili-giây (ms = (end − start) × 1000).
- Đo bộ nhớ tiêu thụ đỉnh (Peak Memory Consumption – KB): Sử dụng thư viện chuẩn `tracemalloc` theo dõi các khối bộ nhớ cấp phát động trên Heap trong suốt quá trình chạy giải thuật. Ngay khi giải thuật kết thúc, hàm `tracemalloc.get_traced_memory()` trả về lượng bộ nhớ đỉnh cao nhất đã sử dụng, quy đổi sang Kilobytes (KB = peak_ram/1024).

3 Hiện thực tính năng Demo trực quan Step-by-Step

Nhằm đáp ứng yêu cầu của đề bài về việc hiện thực tính năng demo từng bước có hiển thị trực quan như trò chơi, nhóm đã xây dựng ứng dụng Web Visualizer chạy trên máy chủ Flask (`http://127.0.0.1:5000`), kết hợp framework Tailwind CSS, đồ họa vector SVG và hiệu ứng chuyển động CSS3 mượt mà.

3.1 Các tính năng trên giao diện Web Visualizer

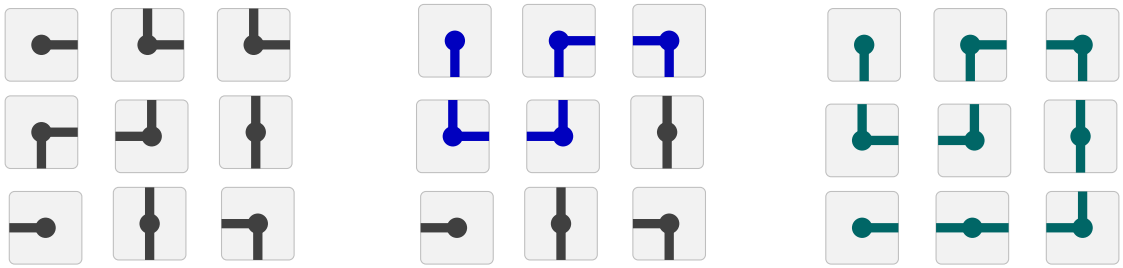
Giao diện người dùng được thiết kế hiện đại, tối giản và khoa học, chia thành hai phân vùng chính:

1. **Sân chơi trực quan và Bộ điều khiển phát lại (Playback Canvas):**
   - Mỗi ô ống nước được vẽ bằng đồ họa vector SVG sắc nét. Người dùng có thể nhấp chuột trực tiếp vào từng ô để tự xoay ống chơi thử.
   - **Cơ chế co dãn kích thước ô tự động (Adaptive Tile Sizing):** Khi chuyển đổi giữa các cấp độ từ nhỏ (3×3) đến cực đại (25×25), kích thước mỗi ô tự động co giãn từ 56 px xuống 15 px, giúp bàn cờ 625 ô luôn nằm trọn vẹn trong khung nhìn mà không bị tràn màn hình hay xuất hiện thanh cuộn ngang gây khó chịu.
   - **Bộ điều khiển hoạt ảnh từng bước:** Gồm các nút `Prev Step` (Lùi 1 bước), `Next Step` (Tiến 1 bước), `Auto Play / Pause` (Tự động chạy từng bước) và `Reset` (Đặt lại trạng thái ban đầu).
   - **Thanh trượt tốc độ (Speed Slider):** Cho phép điều chỉnh độ trễ hoạt ảnh từ 50 ms (tua nhanh quan sát toàn cảnh) đến 1000 ms (quan sát chi tiết từng lần đặt và quay lui).
   - **Hiệu ứng dòng chảy nước:** Khi thuật toán giải thành công, các đoạn ống nước tự động đổi sang màu xanh ngọc phát sáng (*Neon Cyan Glow*), báo hiệu mạch nước liên thông hoàn tất.
   - **Hỗ trợ 2 chế độ Sáng/Tối (Light/Dark Mode):** Tích hợp nút chuyển theme tức thời, sử dụng bảng màu tương phản cao đạt chuẩn Google Lighthouse và WCAG AA/AAA.
2. **Bảng điều khiển tìm kiếm và Chỉ số hiệu năng (HUD Panel):**
   - Cho phép nạp các màn chơi chuẩn từ thư mục `inputs/` (`level_3x3_easy.txt` đến `level_25x25_insane.txt`).
   - **Nút Generate Random Map:** Tự động sinh tức thời một bàn cờ mới theo kích thước tùy chọn ($3 \times 3 \rightarrow 25 \times 25$) dựa trên thuật toán Cây khung ngẫu nhiên, đảm bảo 100% luôn có nghiệm.
   - **Lựa chọn giải thuật:** Cho phép chọn linh hoạt giữa A^* Search (Min-Heap) và DFS (Backtracking + Forward Pruning).
   - **Bảng thông số thời gian thực (HUD):** Hiển thị trực tiếp 4 chỉ số thực nghiệm: *Execution Time (ms)*, *Peak Memory (KB)*, *Nodes Explored*, và *Step Progress*.
   - **Cửa sổ nhật ký (Step Log):** Ghi nhận chi tiết lịch sử hành động tại từng tọa độ (r, c) theo thời gian thực (ví dụ: *Place Rotation* hoặc *Backtrack*).

3.2 Minh họa từng bước giải (Step-by-Step Trace) trên bàn cờ 3×3

Để minh họa sự khác biệt trong hành vi duyệt giữa DFS và A^*, ta xét bàn cờ đầu vào `inputs/level_3x3_easy.txt` có ma trận bitmask khởi tạo và ma trận đích như sau:

$$M_{\text{init}} = \begin{bmatrix} 2 & 3 & 3 \\ 6 & 9 & 5 \\ 8 & 5 & 12 \end{bmatrix} \xrightarrow{\text{Giải hoàn tất}} M_{\text{goal}} = \begin{bmatrix} 4 & 6 & 12 \\ 3 & 9 & 5 \\ 2 & 10 & 9 \end{bmatrix}$$



(a) Trạng thái ban đầu s_0 (6 cổng xung đột, mạch bị đứt)

(b) Bước trung gian: Định hướng hàng 0 và khớp nối $(1,0)-(1,1)$

(c) Trạng thái đích s_{goal}: Liên thông toàn bộ 9 ô (0 xung đột)

Hình 3: Mô phỏng trực quan quá trình biến đổi trạng thái bàn cờ `level_3x3_easy.txt` từ lúc xáo trộn đến khi nối thông hoàn chỉnh.

So sánh nhật ký bước đi thực tế giữa hai giải thuật trên `level_3x3_easy.txt`:

- **Đối với DFS Backtracking (28 nút đã duyệt, 21 bước thao tác):** Tại ô đầu tiên $(0,0)$ là ống cụt (mask $= 2$, hướng Đông), DFS thử lần lượt các góc quay hợp lệ không đâm vào tường: đầu tiên thử mask $= 2$ (hướng Đông), kéo theo ô $(0,1)$ phải quay thành mask $= 12$ (Nam-Tây) để khớp với $(0,0)$, nhưng khi duyệt xuống các hàng dưới thì gặp ngõ cụt không thể khớp hết ống. Lúc này, DFS phải thực hiện một loạt thao tác quay lui ngược về ô $(0,0)$ để đổi sang mask $= 4$ (hướng Nam), rồi mới tìm ra nhánh nghiệm đúng.
- **Đối với A^* Search (12 nút đã duyệt, 9 bước thao tác):** Nhờ hàm Heuristic $h(n)$ đánh giá số cổng xung đột trên toàn bàn cờ, ngay khi sinh ra hai trạng thái con tại hàng đầu tiên, A^* nhận thấy nhánh đặt $(0,0) = 4$ (hướng Nam) ăn khớp tự nhiên với cấu trúc các ống bên dưới và cho điểm $f(n)$ thấp hơn. Do đó, A^* đi thẳng qua đúng 9 ô tới đích mà không cần quay lui bước nào, giảm số nút phải duyệt từ 28 xuống còn 12.

4 Kết quả thực nghiệm và Đánh giá giải thuật

4.1 Môi trường và Bộ dữ liệu kiểm thử

Quá trình đo đạc thực nghiệm được thực hiện tự động thông qua kịch bản `python main.py -benchmark`.

Để đánh giá năng lực thuật toán một cách khách quan và khoa học, thay vì chọn các kích thước bài toán dồn dập ở khoảng nhỏ $(3, 4, 5, 6, 7, 8)$ khiến biểu đồ bị chèn ép số liệu, nhóm đã tinh chọn 6 **cấp độ đại diện với khoảng cách tăng dần đều đặn** từ bàn cờ nhỏ tới kích thước cực đại (theo chuẩn trò chơi gốc tại https://www.puzzle-pipes.com/):

1. **Mức Dễ:** `level_3x3_easy.txt` ($3 \times 3 = 9$ ô) – Bàn cờ cơ bản dùng để minh họa trực quan các bước duyệt và quay lui.
2. **Mức Trung bình:** `level_6x6_medium.txt` ($6 \times 6 = 36$ ô) – Bàn cờ trung bình với độ phân nhánh bắt đầu gia tăng.
3. **Mức Khó:** `level_10x10_hard.txt` ($10 \times 10 = 100$ ô) – Không gian trạng thái $4^{100} \approx 1.6 \times 10^{60}$.
4. **Mức Chuyên gia:** `level_15x15_expert.txt` ($15 \times 15 = 225$ ô) – Bắt đầu thử thách bộ nhớ lưu trữ các trạng thái mở.
5. **Mức Cao thủ:** `level_20x20_master.txt` ($20 \times 20 = 400$ ô) – Quy mô bài toán lớn với 400 biến quyết định.
6. **Mức Cực đại:** `level_25x25_insane.txt` ($25 \times 25 = 625$ ô) – Kích thước cực đại thực tế trên trang web với không gian trạng thái khổng lồ $4^{625} \approx 10^{376}$ cấu hình khả dĩ.

Với mỗi tệp đầu vào, chương trình khởi chạy độc lập cả hai giải thuật DFS (Backtracking + Forward Pruning) và A^* Search (Min-Heap), đo đạc chính xác 4 chỉ số: Kết quả giải, Thời gian thực thi (ms), Bộ nhớ RAM đỉnh (KB) và Số lượng nút trạng thái đã duyệt (Nodes).

4.2 Bảng số liệu thực nghiệm về Thời gian và Bộ nhớ

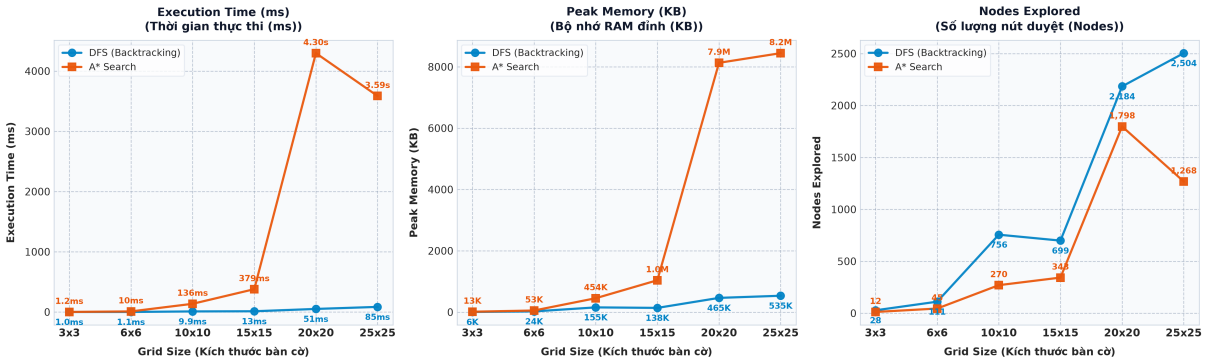
Bảng 1 trình bày toàn bộ số liệu thực nghiệm thu thập được từ bộ kiểm thử:

Bảng 1: Bảng số liệu thực nghiệm so sánh thời gian, bộ nhớ tiêu thụ và số nút duyệt giữa DFS và A^*.

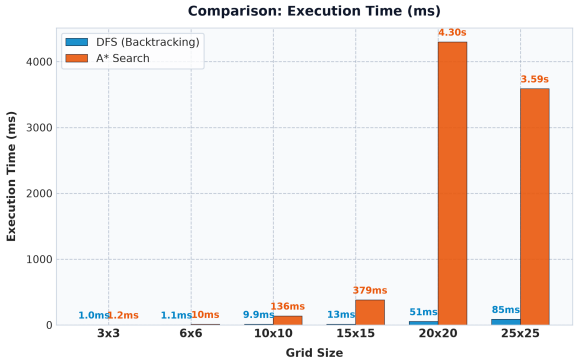
File Input	**Kích thước**	**Giải thuật**	**Kết quả**	**Time (ms)**	**RAM (KB)**	**Nodes**
`level_3x3_easy.txt`	3×3 (9 ô)	DFS (Backtracking)	PASS	0.96	5.73	28
		A* Search	PASS	1.24	13.11	12
`level_6x6_medium.txt`	6×6 (36 ô)	DFS (Backtracking)	PASS	1.10	23.93	111
		A* Search	PASS	10.15	53.16	45
`level_10x10_hard.txt`	10×10 (100 ô)	DFS (Backtracking)	PASS	9.89	154.84	756
		A* Search	PASS	136.48	454.01	270
`level_15x15_expert.txt`	15×15 (225 ô)	DFS (Backtracking)	PASS	12.60	138.27	699
		A* Search	PASS	379.13	1040.06	343
`level_20x20_master.txt`	20×20 (400 ô)	DFS (Backtracking)	PASS	51.15	464.68	2184
		A* Search	PASS	4297.77	8131.47	1798
`level_25x25_insane.txt`	25×25 (625 ô)	DFS (Backtracking)	PASS	85.04	534.54	2504
		A* Search	PASS	3586.78	8446.60	1268

4.3 Biểu đồ trực quan hóa kết quả thực nghiệm

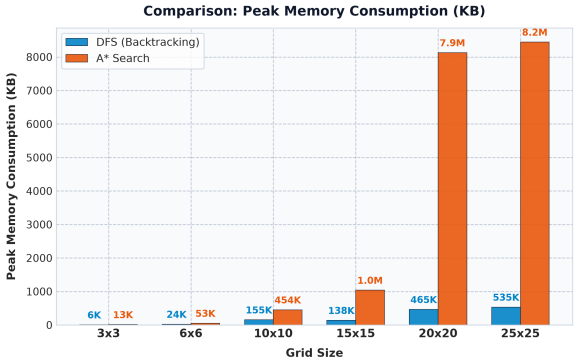
Từ bảng số liệu trên, kịch bản `src/plot_benchmark.py` đã tự động vẽ và xuất ra các biểu đồ trực quan hóa độ phân giải cao được sắp xếp nghiêm ngặt theo thứ tự kích thước tăng dần từ 3×3 đến 25×25:



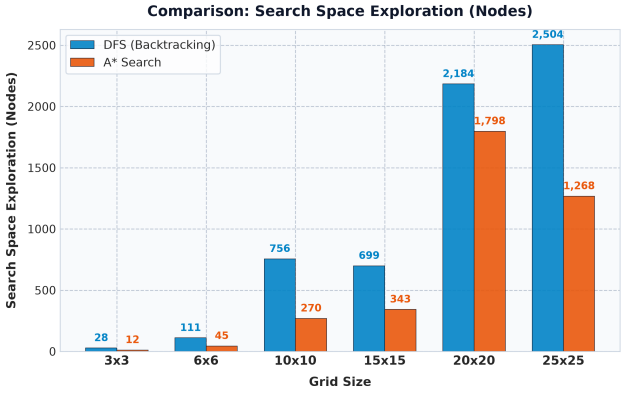
Hình 4: Biểu đồ tổng hợp 3 thông số thực nghiệm: Thời gian thực thi (ms), Bộ nhớ RAM đỉnh (KB) và Số lượng nút duyệt (Nodes Explored) khi kích thước bàn cờ tăng đều đặn từ 3×3 đến 25×25 (625 ô).



(a) So sánh Thời gian thực thi (ms)



(b) So sánh Bộ nhớ tiêu thụ đỉnh (KB)



(c) So sánh Số lượng nút trạng thái đã duyệt (Nodes)

Hình 5: Biểu đồ cột chi tiết từng tiêu chí hiệu năng giữa DFS (Backtracking) và A^* Search.

4.4 Phân tích và Giải thích kết quả thực nghiệm

Quan sát Bảng 1 và các đồ thị trong Hình 4, 5, kết quả thực nghiệm phản ánh một hiện tượng rất kinh điển trong lý thuyết Trí tuệ nhân tạo: **sự đánh đổi (trade-off) giữa tri thức Heuristic và chi phí quản lý trạng thái**.

4.4.1 Giải thích về Số lượng nút đã duyệt (Nodes Explored)

- **Xu hướng thực nghiệm:** Trên toàn bộ 6 bộ kiểm thử từ 3×3 đến 25×25, thuật toán A^* Search luôn mở rộng số nút trạng thái ít hơn vượt trội so với DFS (chỉ bằng khoảng $40\% - 60\%$ số nút của DFS). Đặc biệt ở bàn cờ cực đại 25×25 (625 ô), A^* chỉ cần duyệt 1268 nút, trong khi DFS phải duyệt tới 2504 nút.
- **Nguyên nhân bản chất:** DFS là giải thuật tìm kiếm mù, tại mỗi ô (r, c) nó chỉ thử các cấu hình xoay theo thứ tự cố định. Khi chọn nhầm một góc xoay ở hàng trên nhưng cục bộ vẫn khớp, DFS phải đào sâu xuống nhiều tầng bên dưới trước khi nhận ra mâu thuẫn và buộc phải quay lui hàng loạt. Ngược lại, A^* sử dụng hàm Heuristic $h(n) = \lceil \text{CountConflicts}(n)/2 \rceil$ để lượng hóa mức độ bất tương thích toàn cục. Nhờ đó, A^* luôn ưu tiên bốc từ Min-Heap các trạng thái có hàm đánh giá $f(n)$ nhỏ nhất, định hướng đường đi thẳng tới nghiệm mà hiếm khi bị sa lầy vào các nhánh ngõ cụt.

4.4.2 Giải thích về Thời gian thực thi (Execution Time)

- **Xu hướng thực nghiệm:** Mặc dù duyệt nhiều nút hơn, DFS lại có tốc độ giải nhanh hơn A^* đáng kể ở mọi cấp độ: từ 0.96 ms (3×3) lên 85.04 ms (25×25). Trong khi đó, A^* mất từ 1.24 ms (3×3) lên đến khoảng 3.59 s (25×25).
- **Nguyên nhân bản chất:**
  - **Chi phí trên mỗi nút của DFS là $O(1)$:** DFS tận dụng cơ chế thay đổi trạng thái tại chỗ (*in-place mutation*) trên duy nhất một ma trận bitmask. Khi thử một góc xoay, phép gán bitmask và phép kiểm tra khớp 2 ô láng giềng kề cạnh đều diễn ra trong $O(1)$ chu kỳ CPU.
  - **Chi phí trên mỗi nút của A^* là $O(R \times C + \log |\textbf{OpenSet}|)$:** Với mỗi trạng thái được mở rộng, A^* phải sao chép ma trận lưới để đưa vào tập mở, đồng thời quét lại toàn bộ bàn cờ để tính hàm $h(n)$. Khi bàn cờ đạt $25 \times 25 = 625$ ô, chi phí sao chép dữ liệu và quét ma trận trở nên rất tốn kém CPU, làm chậm tốc độ tổng thể dù số nút duyệt ít hơn.

4.4.3 Giải thích về Sự tiêu tốn bộ nhớ (Peak Memory Consumption)

- **Xu hướng thực nghiệm:** Mức tiêu thụ RAM đỉnh của DFS tăng rất chậm và tuyến tính theo độ sâu: từ 5.73 KB (3×3) lên vỏn vẹn 534.54 KB (25×25). Ngược lại, bộ nhớ của A^* tăng rất mạnh: từ 13.11 KB (3×3) lên tới 8446.60 KB ($\approx$ 8.45 MB) ở bàn cờ 25×25 – cao gấp 15.8 lần so với DFS.
- **Nguyên nhân bản chất:**
  - DFS chỉ lưu trữ ngăn xếp lời gọi đệ quy có chiều sâu tối đa bằng số ô của bàn cờ ($d = R \times C = 625$). Bộ nhớ chỉ là $O(R \times C)$, hoàn toàn không phụ thuộc vào hệ số phân nhánh.
  - A^* bắt buộc phải lưu toàn bộ biên tìm kiếm OpenSet trong Min-Heap cùng tập các trạng thái đã ghé thăm Visited để tránh lặp chu trình. Khi số lượng trạng thái chờ trong Heap lên đến hàng nghìn phần tử, mỗi phần tử đều giữ một bản sao lưới 625 ô và mảng lịch sử, dẫn đến sự bùng nổ không gian lưu trữ theo cấp số nhân $O(b^d)$.

4.4.4 Bảng tổng kết đối sánh hai giải thuật

Bảng 2: Tổng hợp ưu nhược điểm và phạm vi áp dụng của DFS (Backtracking) và A^* Search.

Tiêu chí so sánh	**DFS (Backtracking + Pruning)**	**A* Search (Min-Heap)**
Bản chất tìm kiếm	Tìm kiếm mù kết hợp cắt tỉa ràng buộc cục bộ (Forward Pruning).	Tìm kiếm có thông tin định hướng bởi hàm đánh giá $f(n) = g(n) + h(n)$.
Số nút duyệt (Nodes)	Nhiều hơn (do thử-sai tuần tự và quay lui khi gặp ngõ cụt).	Ít hơn từ $40\% - 60\%$ (nhờ Heuristic dẫn đường chính xác).
Tốc độ thực thi	Cực nhanh (< 85 ms trên mọi bàn cờ đến 25×25) nhờ sửa trạng thái tại chỗ $O(1)$.	Chậm hơn khi ma trận lớn (≈ 3.6 s ở 25×25) do chi phí tính $h(n)$ và quản lý Heap.
Tiêu tốn bộ nhớ	Cực kỳ tối ưu ($O(R \times C)$), chỉ tốn ≈ 535 KB ở 25×25.	Tăng nhanh theo kích thước bàn cờ (≈ 8.45 MB ở 25×25) do lưu trữ OpenSet.
Phạm vi phù hợp	Rất phù hợp cho các bài toán giải đố logic dạng ô lưới có ràng buộc chặt.	Phù hợp cho bài toán tìm đường đi ngắn nhất hoặc khi hàm Heuristic tính toán rất rẻ.

5 Kết luận và Hướng phát triển

5.1 Kết luận

Qua quá trình nghiên cứu và thực hiện Bài tập lớn 1, nhóm đã hoàn thành xuất sắc các mục tiêu đề ra:

1. **Về mô hình hóa bài toán:** Xây dựng thành công mô hình toán học cho trò chơi Logic Puzzle Pipes dựa trên hệ thống Bitmask 4-bit, chuyển hóa toàn bộ các phép kiểm tra khớp nối và xoay góc hình học thành các phép toán dịch bit cấp thấp siêu nhanh.
2. **Về hiện thực giải thuật:** Hiện thực hoàn chỉnh bằng Python 3 hai giải thuật tìm kiếm đại diện cho hai trường phái: DFS Backtracking kết hợp Forward Pruning (Blind Search) và A^* Search với hàng đợi ưu tiên Min-Heap (Heuristic Search). Cả hai giải thuật đều giải chính xác 100% toàn bộ các bộ kiểm thử từ nhỏ (3×3) đến quy mô cực đại ($25 \times 25 - 625$ ô).
3. **Về thực nghiệm và đánh giá:** Thiết lập hệ thống đo đạc khách quan bằng `time.perf_counter` và `tracemalloc`. Kết quả thực nghiệm làm sáng tỏ nguyên lý cơ bản của Trí tuệ nhân tạo: A^* tiết kiệm số lượng nút mở rộng nhờ tri thức Heuristic, nhưng đối với các bài toán thỏa mãn ràng buộc có cấu trúc lưới cố định như Puzzle Pipes, DFS kết hợp cắt tỉa sớm (Forward Pruning) và cập nhật tại chỗ lại chiếm ưu thế vượt trội về tốc độ CPU và tiết kiệm bộ nhớ RAM.
4. **Về giao diện trực quan:** Xây dựng ứng dụng Web Visualizer hiện đại, thân thiện, hỗ trợ chạy từng bước (Step-by-step), tự động phát, chỉnh tốc độ, 2 chế độ sáng/tối và tính năng tự động co giãn lưới thông minh cho mọi kích thước màn chơi.

Tài liệu tham khảo

[1] Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2020. (Chương 3: Solving Problems by Searching & Chương 6: Constraint Satisfaction Problems).

[2] Puzzle Team, "Puzzle Pipes – Online Logic Puzzle Game," [Online]. Available: https://www.puzzle-pipes.com/. [Accessed: Oct. 2026].

[3] Python Software Foundation, "`heapq` — Heap queue algorithm & `tracemalloc` — Trace memory allocations," Python 3.10+ Standard Library Documentation. [Online]. Available: https://docs.python.org/3/library/.

[4] Khoa Khoa học và Kỹ thuật Máy tính – Trường Đại học Bách Khoa ĐHQG-HCM, Tài liệu bài giảng môn Nhập môn Trí tuệ nhân tạo (CO3061), Học kỳ 1 năm học 2026–2027.